

《数据结构与算法设计》

课程设计总结

学号： 2353900

姓名： 汪嘉晨

专业： 计算机科学与技术

2025 年 9 月

目录

第一部分 算法实现设计说明	3
1.1 题目	3
1.2 软件功能	3
1.2.1 功能	3
1.2.2 实现方式	4
1.3 设计思想	4
1.3.1 数据结构	4
1.3.2 核心功能	4
1.3.3 可视化思想	5
1.4 逻辑结构与物理结构	5
1.4.1 逻辑结构	5
1.4.2 物理结构	6
1.5 开发平台	6
1.5.1 开发语言/框架	6
1.5.2 第三方库	6
1.6 系统的运行结果分析说明	7
1.7 操作说明	10
第二部分 综合应用设计说明	11
2.1 题目	11
2.2 软件功能	11
2.3 设计思想	12
2.3.1 数据结构的选择与理由	12
2.3.2 复杂度与正确性分析	12
2.4 逻辑结构与物理结构	12
2.4.1 逻辑结构	12
2.4.2 逻辑数据结构	13
2.4.3 物理结构	13
2.5 开发平台	13
2.6 系统的运行结果分析说明	14
2.6.1 应用开发工具与调试过程	14
2.6.2 运行成果	14
2.7 操作说明	16
第三部分 实践总结	17
3.1.所做的工作	17
3.2.总结与收获	17
第四部分 参考文献	18

第一部分 算法实现设计说明

1.1 题目

0. 试从空树出发构造一棵深度至少为 3（不包括失误节点）的 3 阶 B 树（又称 23 树），并可以随时进行查找、插入、删除等操作。

要求：能够把构造和删除过程中的 B 树随时显示输出来，能给出查找是否成功的有关信息。

1.2 软件功能

本项目实现了题目要求“从空树出发构造一棵深度至少为 3 的 3 阶 B 树（23 树），支持随时查找、插入、删除并可视化显示和反馈”。项目包含 Python 后端（Flask 提供 API、实现 B 树算法）与前端（HTML/CSS/JS 可视化交互）。

1.2.1 功能

1. 插入（Insert）/ 删除（Delete）/ 查找（Search）/ 清空（Clear）键值

从交互上，用户在输入框填入 1-999 的整数后点击对应按钮即可触发。插入会进行去重校验，若键已存在则返回明确提示；合法插入会在叶节点完成并在必要时触发自底向上的分裂与根提升。删除首先在内部节点场景通过前驱替换将问题下沉至叶子，再根据下溢情况执行向兄弟借键或与兄弟合并的再平衡，直至恢复约束并在必要时进行根降级。查找沿着节点内有序键进行二分式区间定位，命中即返回 found 与相对深度；未命中在叶子处终止。清空操作会重置根节点并清除全部结构。所有操作均记录到历史队列并驱动页面的统计信息刷新与可视化重绘。

2. 实时可视化树结构（DOM + SVG 连线）

可视化将后端返回的层级结构映射为页面上的节点盒，并按 level 将各层节点分层布局；父子关系通过一个覆盖在内容层之上的 SVG 画布绘制连接线，连接点取父盒底部中心与子盒顶部中心，以保持视觉整洁。渲染采用“重绘整个树 + 按层绘制连线”的策略，能在结构变化后稳定反映当前状态；在插入、查找、删除等操作中，还会对目标键或相关节点短时高亮，以强调关键步骤。

3. 显示操作结果与提示信息（成功/失败、深度、统计信息）

页面顶部的信息区域用于集中反馈操作结果，区分 success / error / info 三种风格。插入与删除会返回简明消息并在必要时附带“由于分裂/借用/合并导致的结构调整”等说明；查找则汇报“是否命中”与命中的层级深度。提示信息会在几秒后自动隐去，避免干扰后续操作。

4. 显示操作历史（时间戳、操作类型、键、结果与细节）

系统维护一条时间序列的历史记录，记录字段包括时间戳、操作类型（insert/search/delete/clear）、目标键（若有）、结果布尔以及补充细节。

5.基本结构验证（有效性标识、统计信息）

验证功能提供对当前树结构健康度的快速判断。后端 ``/api/validate`` 用于返回“结构有效”与统计摘要；前端还提供 ``validator.js`` 的本地校验逻辑，可在需要时检查典型 B 树不变式，如节点键序有序性、内部节点子树数量等于键数+1、所有叶子在同一深度、父子引用一致性以及键值范围约束等。

1.2.2 实现方式

1.后端（``btree_backend.py``）:

- ① ``BTree`/`BTreeNode`` 实现 3 阶 B 树的插入、分裂、删除、借用、合并、搜索等核心算法；
- ② 维护 ``operation_history``，记录每次操作的概要与时间；
- ③ 提供 ``get_tree_structure()`` 生成层级化结构用于前端可视化；
- ④ 统计接口：``get_depth()``、``get_node_count()``、``get_key_count()``。

2.API（``app.py`` via Flask）:

- ① 端点：``/api/insert``、``/api/delete``、``/api/search``、``/api/clear``、``/api/structure``、``/api/history``、``/api/history/clear``、``/api/validate``；
- ② 返回 JSON，包含操作结果、消息、树结构与统计信息。

3.前端:

- ① ``btree_api.js``：封装与后端的 HTTP 调用（fetch）。
- ② ``visualization.js``：将树结构按层分组渲染为节点盒与 SVG 连线；支持动画高亮。
- ③ ``main.js``：界面交互逻辑（读取输入、绑定按钮、消息提示、更新统计、历史显示等）。
- ④ ``validator.js``：可用于离线校验 B 树属性（用于开发调试与扩展）。

1.3 设计思想

1.3.1 数据结构

树的基本单元为节点，节点内部维护一个有序整数数组作为键集合，以及一个与“键槽”对齐的子节点数组。对于 3 阶 B 树（23 树），每个节点可包含 1 或 2 个键；内部节点的子节点数等于键数加一，因此可能是 2 个或 3 个子节点。所有叶子必须处于同一深度，以保证从根至任意叶子的路径长度一致。根节点在空树与极小规模时允许处于退化形态（如仅含一个子树或本身为叶）。

1.3.2 核心功能

查找采用自顶向下的有序定位策略：在当前节点内按序比较键，确定目

标应当落入的区间；若命中则返回，否则递归至相应子节点。到达叶子仍未命中即判定失败。

插入首先进行查重，避免破坏集合语义。当树为空时直接创建根叶并插入；一般情形下，自顶向下定位到叶子并插入到有序位置。若插入后叶子键数达到 3，触发一次分裂：以中间键作为提升键产生右兄弟，并将提升键插入父节点、右兄弟插入到父节点的孩子数组中。若父节点因此也达到容量上限，则继续向上进行相同的分裂与提升，直至根；当根分裂时，生成新的根节点并使树高加一。

删除在命中内部节点时，先以左子树的最大键替换目标键，从而把删除问题下沉到叶子进行；在叶子成功移除后，如出现键数为 0 的下溢，需要先尝试从兄弟节点“借键”：若左兄弟或右兄弟含有 2 个键，可以通过旋转父键与兄弟最靠近的一侧键来填补当前叶子的空位，同时在非叶情形下相应移动一个子指针，以保持子树数量与范围约束一致。若两侧兄弟都无法借，则与其中一侧兄弟合并：将父节点位于两者之间的分割键下移，与兄弟的键合并到一个节点中，并删除父节点对应的键与子指针；如果父节点因此下溢，则递归向父继续借用或合并，直至根。删除完成后若根节点变为空且存在唯一子树，需要将该子树提升为新的根以避免额外的一层高度。

为提升可维护性与可诊断性，核心功能在每一步关键结构变化处记录简要的操作摘要写入 ``operation_history``，并在 API 层返回必要的统计信息与最终树结构。

1.3.3 可视化思想

可视化设计遵循“数据驱动、分层渲染、轻量动画”的原则。后端返回的层级化结构是单一数据源，前端首先按 `level` 将节点分组，自上而下生成每一层的节点盒，节点盒中以紧凑的水平布局展示 1 或 2 个键。为了准确表现父子关系，在节点盒层之下铺设一层 SVG 画布，读取 DOM 的几何信息（盒子中心点与上下边界）后绘制连接线，线条采用圆角端点与中性配色以降低视觉干扰。渲染采用“整体重绘 + 延迟连线”的两步法：先生成 DOM 再在一次布局完成后绘制 SVG 线，必要时触发一次强制重绘确保连线位置与盒子对齐。

1.4 逻辑结构与物理结构

1.4.1 逻辑结构

后端以 `BTreeNode` 与 `BTree` 为核心。`BTreeNode` 表达 3 阶 B-树的基本节点形态，内部用升序数组保存 1 至 2 个键，并在内部节点时以“键数+1”的方式维护 2 至 3 个子指针；它携带 `is_leaf` 标志与 `parent` 引用，以便在删除或分裂后的向上修复中保持父子关系一致。节点原语涵盖在单节点内查找键、定位插入位置、插入/移除单键，以及在键数临时达到 3 时将中间键

上推并产生右兄弟的分裂操作。BTree 则封装全树行为，持有根指针并实现 search、insert 与 delete 的完整流程；插入在叶子落位后可能自底向上级联分裂，删除在内部命中时通过前驱替换下沉到叶子并在下溢时先尝试从兄弟借键、再退而合并，必要时调整根高度。为配合前端展示，树可以导出仅含值与层级信息的层级化结构，并提供深度、节点数与键总数的统计。同时，树在每次用户操作后记录时间戳、操作类型、目标键与结果细节，形成可回放的 operation_history。

接口层通过 Flask 提供与前端对齐的 REST 契约。插入、删除、查找与清空使用 POST，并在需要时携带 {key:int} 的 JSON 负载；结构查询、历史查询与结构验证使用 GET。服务在成功时返回明确的消息、最新的树结构与统计数据，在参数缺失或运行异常时以统一格式返回错误信息，便于前端直接提示用户。

前端由若干职责单一的模块协作完成。btree_api.js 以类 BTreeAPI 统一封装与后端的网络通信，并提供与树操作同名的方法；在此之上，一个轻量级的前端 BTree 包装类同步本地的操作历史，方便历史面板即时刷新。visualization.js 负责把后端导出的层级结构渲染为节点盒，并通过一个覆盖层的 SVG 在父节点与子节点之间绘制连线；在插入、查找与删除期间，它用短时高亮与轻量动画强调关键节点，并在布局完成后强制重绘连线以确保几何对齐。main.js 承担应用控制器角色，完成输入校验、按钮状态切换、消息提示与统计面板刷新，并协调可视化的渲染与动画。validator.js 则提供本地的结构一致性校验逻辑，可与后端返回的验证结果互证，提升调试效率。

1.4.2 物理结构

物理层面，后端由 app.py 与 btree_backend.py 构成：前者负责启动 Flask、注册路由并聚合 BTree 实例，后者承载 3 阶 B-树的全部算法、结构导出与统计逻辑。前端的入口页面为 btree.html，它加载 style.css 提供整体观感与节点/连线的可视化样式；btree_api.js 实现接口调用与错误包装，visualization.js 实现 DOM+SVG 的结构呈现与动画，main.js 组织页面事件与状态同步，validator.js 则用于本地规则校验。

运行依赖方面，后端仅需 Python 3.8+、Flask 与 flask-cors，即可在本机 127.0.0.1:5000 提供 JSON 接口；前端不依赖框架，使用原生 HTML/CSS/JS，即便直接从文件系统打开也能通过 CORS 访问本地 API。

1.5 开发平台

1.5.1 开发语言/框架

Python（其中安装了 Flask 与 flaskcors 库），HTML/CSS/JavaScript

1.5.2 第三方库

Flask（Web 框架）、flaskcors（跨域支持）

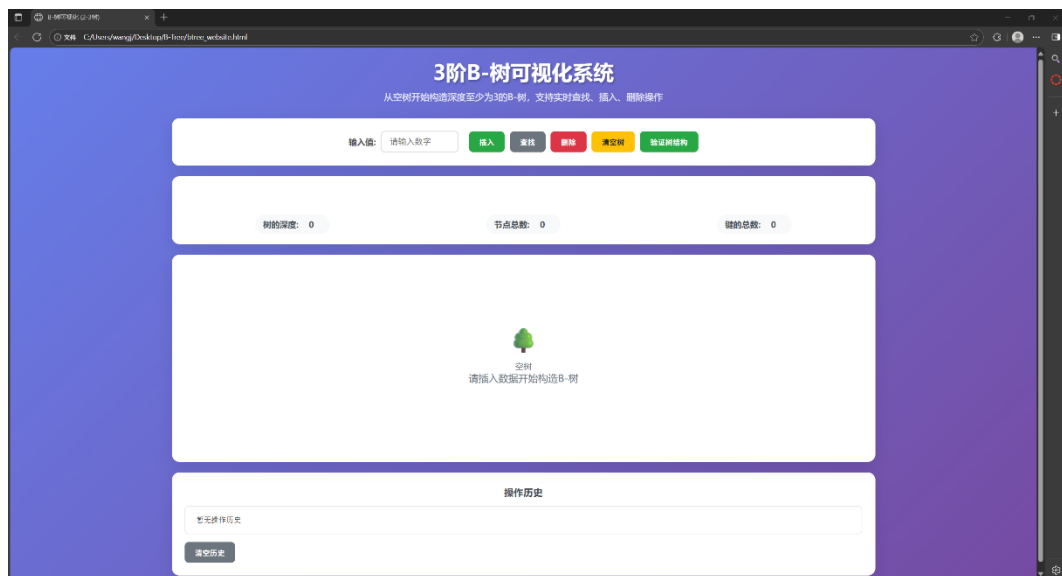
1.5.3 操作方法

1.6 系统的运行结果分析说明

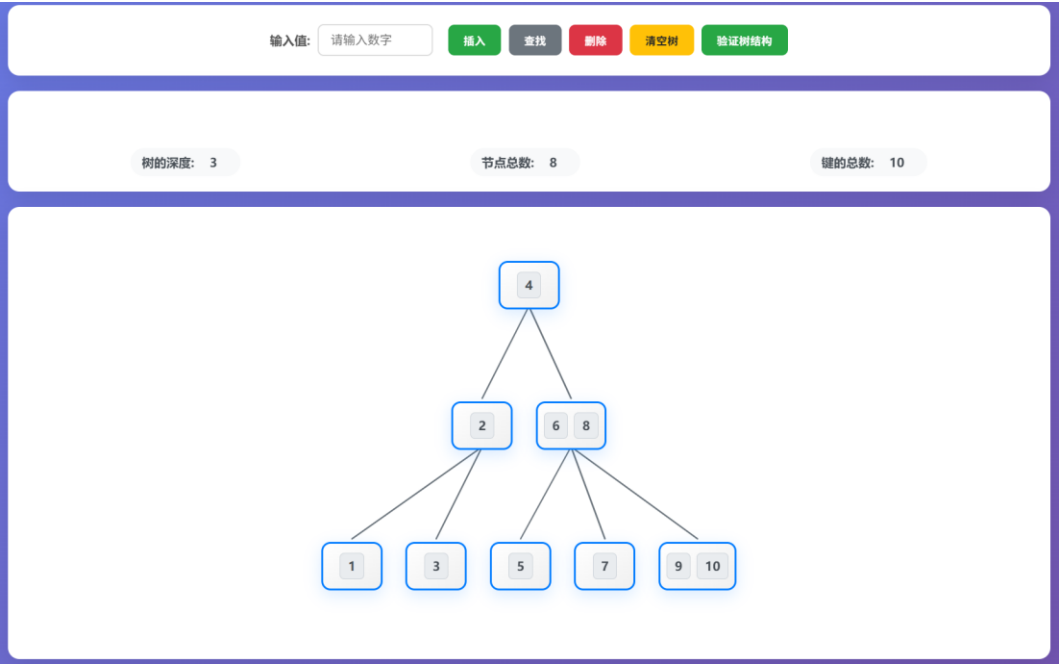
首先点击 start_backend.bat 打开后端

```
C:\WINDOWS\system32\cmd.exe
POST /api/insert - 插入键值
POST /api/search - 查找键值
POST /api/delete - 删除键值
POST /api/clear - 清空树
GET /api/structure - 获取树结构
GET /api/history - 获取操作历史
POST /api/history/clear - 清空历史
GET /api/validate - 验证树结构
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://100.81.23.103:5000
Press CTRL+C to quit
* Restarting with stat
启动B-Tree Backend API服务...
访问 http://localhost:5000 查看API状态
前端可以通过以下端点与后端通信:
POST /api/insert - 插入键值
POST /api/search - 查找键值
POST /api/delete - 删除键值
POST /api/clear - 清空树
GET /api/structure - 获取树结构
GET /api/history - 获取操作历史
POST /api/history/clear - 清空历史
GET /api/validate - 验证树结构
* Debugger is active!
* Debugger PIN: 121-694-546
```

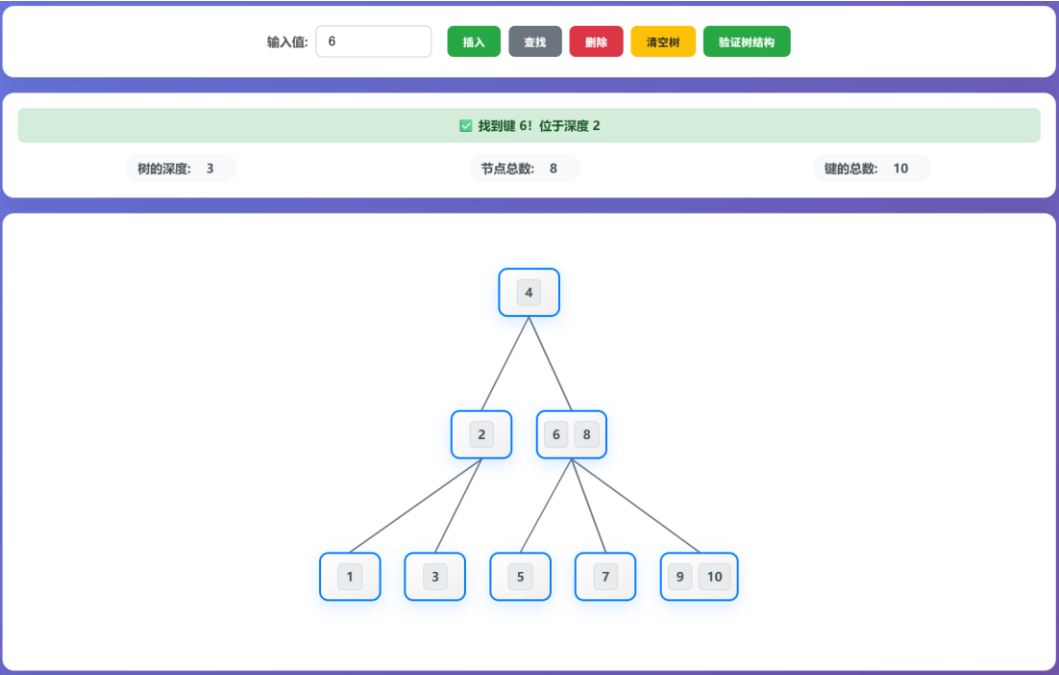
接着打开 btree_website.html，进入程序主界面，上面显示了标题，中间显示了具体操作“插入”“查找”“删除”“清空”“验证树结构”。



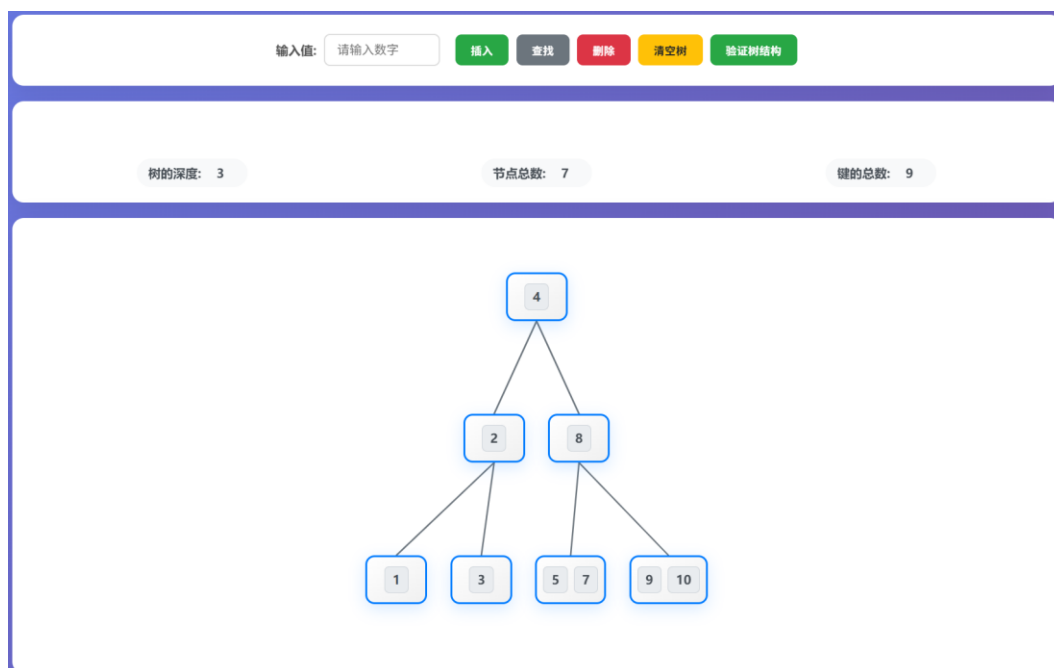
具体操作如图,插入数字 1-10:



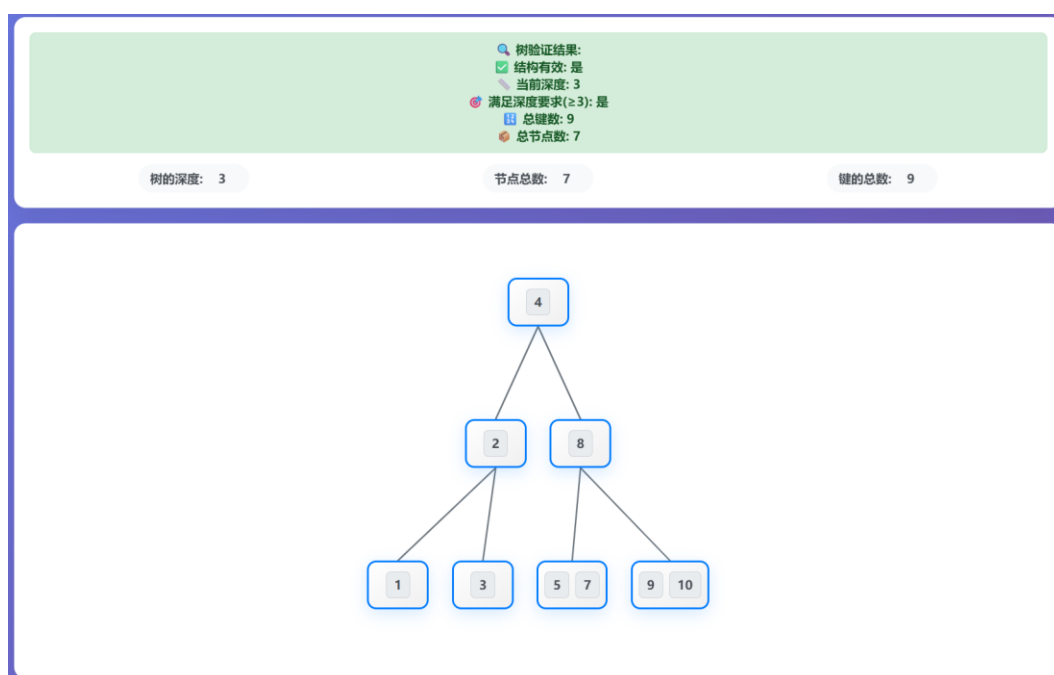
查找数字 6:



删除数字 6:



验证树的结构:



在页面下端还显示了最近执行操作:

操作历史		
19:36:53	删除 6	删除成功
19:36:51	查找 6	在深度 2 找到
19:36:26	查找 6	在深度 2 找到
19:35:54	插入 10	插入成功
19:35:51	插入 9	插入成功
清空历史		

综上, 程序完成了“从空树出发构造一棵深度至少为 3 (不包括失误节点) 的 3 阶 B-树 (2-3 树), 支持随时查找、插入、删除”的要求。

1.7 操作说明

1.启动后端:

在工程根目录执行 PowerShell:

```
pip install flask flaskcors
```

```
python app.py
```

看到控制台提示“BTree Backend API is running!”即服务就绪。

2.打开前端:

通过浏览器打开 `btree.html`; 页面顶部显示标题与操作按钮。

3.基本操作:

在“输入值”中输入不大于 1000 的正整数;

插入: 点击“插入”, 若成功会绿色提示并高亮新插入键;

查找: 点击“查找”, 找到则高亮该键并提示所在深度;

删除: 点击“删除”, 会先短暂高亮目标键再执行删除与重绘;

清空: 点击“清空树”恢复为空;

验证: 点击“验证树结构”查看“结构有效”“当前深度”“是否满足 ≥ 3 ”等;

页面下方“操作历史”滚动显示最近 20 次操作, 支持清空历史。

4.统计信息:

页面中部展示“树的深度 / 节点总数 / 键的总数”, 随操作自动刷新。

5.附: 关键接口

```
POST `/api/insert` {key}
```

```
POST `/api/delete` {key}
```

```
POST `/api/search` {key}
```

```
POST `/api/clear`
```

```
GET `/api/structure`
```

```
GET `/api/history`
```

```
POST `/api/history/clear`
```

```
GET `/api/validate`
```

第二部分 综合应用设计说明

2.1 题目

0.在关系数据库中，所有数据对象都以表的形式存储，如需在关系数据中存储树结构，需要设置一指向父节点的属性来实现，该过程可以通过如下线性表节点的存储结构模拟：

```
struct Node
{
    int id;//该线性表中所有结点的 ID 都唯一
    ElemType data;//结点的值，自定义数据类型
    int pid;//表示父节点，值为 0 表示根节点，对应线性表中其他节点的 id 值
    Node *next;//指向线性表下一节点
}
```

(1) 请设计一算法，根据 pid 的指向，将该线性表中存储的节点转换为树形结构

(2) 在树结构中插入一节点，自动将其插入到线性表中

(3) 在树结构中删除一节点，自动更新线性表的结构

2.2 软件功能

一是数据管理功能。系统使用自增 ID 确保每条记录的唯一性，通过内置哈希索引维护 id 到记录的快速映射，使父节点校验、删除定位等常用操作具备常数时间的访问能力。插入数据时会校验父 ID 是否存在（pid=0 表示根层插入，无需校验），失败则拒绝写入并返回明确错误原因。删除数据时采用级联策略：以目标 ID 为根，递归删除其所有子孙记录，保证线性表不残留孤儿节点，树重构后相应分支整体消失。系统对空表、单节点、深层嵌套、多分支等常见形态均能正确处理。

二是线性表到树的构建功能。系统在读取树形数据时，先线性遍历表构建所有节点对象，再按 pid 挂接父子关系，最终得到一棵以 pid=0 记录为根的树。该过程时间复杂度 $O(n)$ 、空间复杂度 $O(n)$ ，适合数据的快速重建。若线性表中存在多条 pid=0 的记录，当前实现以最后一次遇到的根为返回根（同一数据集建议只保留一个根以避免语义歧义）。

三是增删接口功能。插入接口接收父 ID 与节点数据，成功时返回新节点 ID；删除接口接收目标 ID 并执行级联删除，删除不存在的 ID 会返回失败标志。两类接口均为无副作用的简单契约：输入明确、校验严格、返回结构清晰，便于通过任何 HTTP 客户端或脚本进行自动化测试。与此同时，查询接口提供线性表原貌与树形派生两种视图，前者便于核对存储状态，后者

便于直观看层级关系。

2.3 设计思想

2.3.1 数据结构的选择与理由

1. 自引用父指针 (Adjacency List) 以关系数据库最常见的自引用模式表示树，每条记录携带 `pid` 指向父节点，`pid=0` 表示根。选择该模型的理由是直观、易于维护、插入/删除语义清晰，且与题目要求完全一致。在教学与实验环境中，其可读性与可验证性更优于更复杂的替代模型（比如嵌套集合、物化路径、闭包表等）。

2. 哈希索引 (`id_index: dict[id -> record]`) 在内存中维护 `id` 到记录的映射，能以 $O(1)$ 的时间完成父节点存在性校验与删除时的定位，加速高频操作。与仅遍历列表相比，索引显著降低了插入与删除的时间开销，使插入达到 $O(1)$ 、删除以被删子树规模计的线性复杂度。

3. 树节点对象 (Node) 读取树时用临时 Node 对象承载 `id`、`data`、`pid` 与 `children` 数组，便于进行递归/迭代遍历、序列化输出等操作。Node 不参与持久化，只在树视图构建过程中存在，避免了写时同步多份结构带来的复杂性。

2.3.2 复杂度与正确性分析

线性表→树重建是 $O(n)$ ，插入 $O(1)$ ，删除 $O(k)$ 。这与实验诉求的“直观高效”高度契合。正确性来自三个方面：插入前校验父节点存在性阻止悬挂节点；删除采用递归级联保证不留孤儿；树结构每次读取时重建，避免了写时同步两份结构的不一致风险。只要线性表不包含环，重建的结果就是唯一且正确的树。

2.4 逻辑结构与物理结构

2.4.1 逻辑结构

1. 数据事实层（线性表）

以记录集合表达树结构的“事实存储”。每条记录包含 `id`、`data`、`pid`，`pid=0` 表示根层。该层的角色是唯一事实来源，所有变更（插入、删除、父子关系调整）均首先且仅作用于此，保证写入路径简单可控。

2. 派生视图层（树结构）

在读取时由线性表即时重建得到的一棵树。该层不做持久化，只作为展示、遍历、序列化的运行时对象。它体现“读写分离”的思想：写入不维护树，读取再用线性算法派生，避免双写造成的不一致。

3. 快速访问索引层（哈希索引）

维护 `id` 到记录的映射，服务于父节点存在性校验、目标节点快速定位

与级联删除路径的加速。该层承载“常数时间校验”的质量目标，使插入为 $O(1)$ 、删除定位更直接。

4.服务契约层（接口语义）

提供“查询线性表”“查询树”“插入节点”“删除节点”的最小契约。契约包含明确的输入、校验规则、失败原因与返回结构，确保可测试、可回归。逻辑上保证：插入仅在父存在或 $pid=0$ 时成功；删除支持级联；查询永远基于当前表状态重建树。

2.4.2 逻辑数据结构

1.线性表记录 每条记录满足 id 唯一； pid 为 0 或者指向另一条记录的 id ； $data$ 为任意用户数据。表层的不变式是：不存在自指针（ $pid \neq id$ ）；理想情况下不存在环（若出现环会破坏树语义）。

2.树节点对象 包含 id 、 $data$ 、 pid 和 $children$ 。 $children$ 为顺序列表，保证遍历顺序稳定、易于文本化输出与 JSON 序列化。树节点仅在读取树视图时创建，生命周期短于表数据。

3.索引结构 id_index 是字典映射，满足 $O(1)$ 的查找语义； $next_id$ 为单调递增的计数器，保证新插入记录的 id 唯一且不回退。索引需与线性表保持强一致（插入时写入索引，删除时同步移除索引）。

2.4.3 物理结构

1.文件组织

主程序位于 `database.py`，包含数据模型、存储管理、算法实现与服务接口的整合，便于课程环境下集中阅读与调试。若进行工程化，可按模块拆分为 `models`、`store`、`services` 等。

2.核心类与方法 Node

树节点的运行时模型，字段为 id 、 $data$ 、 pid 、 $children$ ，并提供 `to_dict` 序列化方法。TreeDatabase：线性表与索引的管理者。主要方法包括：— `add_node_to_table`：追加记录并更新索引，返回新 id 。— `remove_node_from_table`：对指定 id 执行递归级联删除，更新表与索引。— `update_node_parent`：更新记录的 pid （预留移动节点能力）。— `linear_to_tree`：两遍法从表构建树节点结构。— `insert_tree_node`：对外插入接口，封装校验与追加。— `delete_tree_node`：对外删除接口，封装级联删除。— `get_table_data` / `get_tree_data`：只读查询，分别返回表与树的派生结果。

2.5 开发平台

实验在 Python 3.11.9 上完成，后端框架使用 Flask 2.3.3，并使用 Flask-CORS 4.0.0 以支持跨源访问。依赖通过 `requirements.txt` 声明，便于安装与复现实验。系统以内存为存储介质，重启即回到初始化演示数据。

2.6 系统的运行结果分析说明

2.6.1 应用开发工具与调试过程

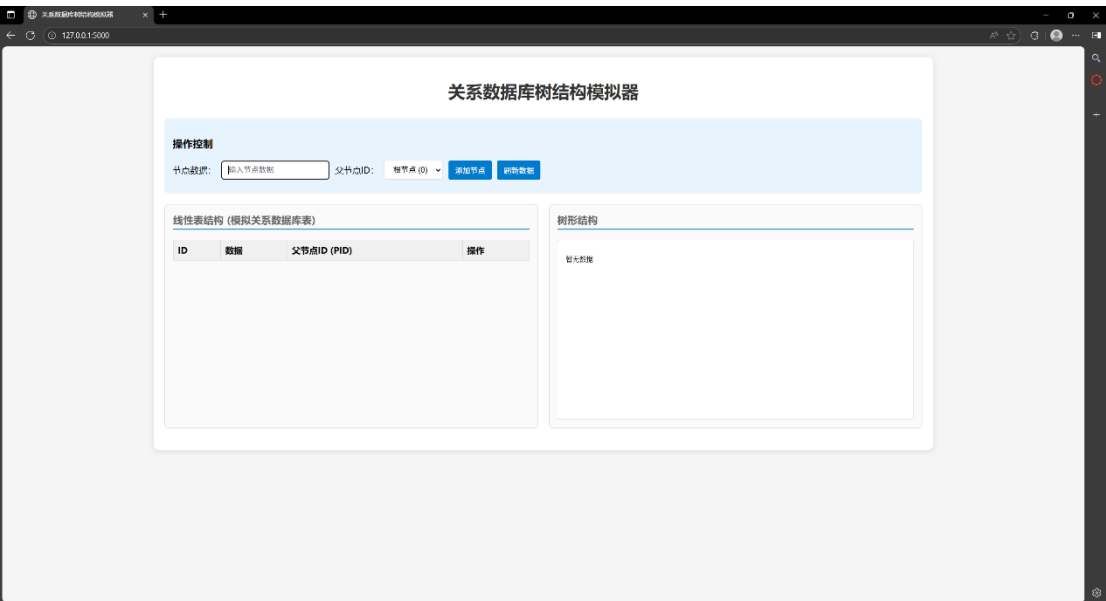
本项目采用 Python+Flask 的轻量级 Web 开发栈进行构建。开发环境为 Windows 10 配合 Python 3.11.9，使用 Visual Studio Code 作为主要编辑器，借助其 Python 扩展提供语法高亮、错误检测和调试支持。Flask 框架的内置开发服务器提供了热重载功能，代码修改后自动重启，大大提升了开发效率。

调试过程主要分为三个阶段。第一阶段是核心算法验证，通过在 TreeDatabase 类中添加 print 语句追踪线性表到树的转换过程，确保两遍扫描算法正确建立父子关系。第二阶段是接口联调，使用 Postman 和 PowerShell 的 Invoke-RestMethod 工具测试 REST API 的输入输出，验证 JSON 序列化、错误处理和状态码返回的正确性。第三阶段是前端集成测试，通过浏览器开发者工具监控网络请求、JavaScript 控制台输出和 DOM 变化，确保前后端数据流畅通无阻。

在开发过程中遇到的主要问题包括：HTML 模板字符串引号缺失导致语法错误，通过仔细检查字符串定界符得以解决；CORS 跨域问题影响前端 Ajax 请求，通过引入 Flask-CORS 扩展妥善处理；递归删除的深度限制问题，虽然在当前测试数据规模下未触发，但已在设计中考虑改为迭代实现的可能性。整个调试过程遵循"单元测试→集成测试→端到端测试"的渐进式验证策略，确保每个模块的功能正确性。

2.6.2 运行成果

首先双击 start_server.bat 并打开 <http://127.0.0.1:5000>，进入程序

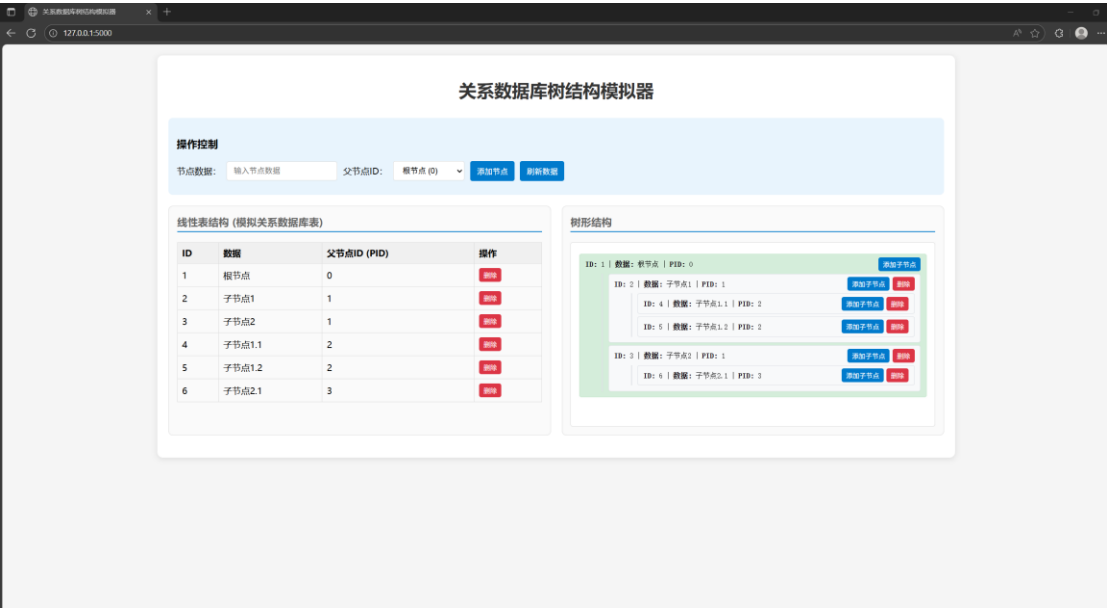


添加节点可以输入节点数据并选择当前已有节点作为父节点

对于树结构，见页面的右半部分，线性表到树的转换算法严格按照 pid 关系建立父子连接，经过多组测试数据验证，从简单的单层树到复杂的多层嵌套结构，都能正确还原层次关系。

2.7 操作说明

- 1.安装 Python 3.11 与依赖包 Flask、Flask-CORS
- 2.双击 start_server.bat
- 3.打开 http://127.0.0.1:5000
- 4.进行具体操作



第三部分 实践总结

3.1.所做的工作

本次课程设计我完成了两个核心模块的开发工作。在关系数据库树结构模拟器方面，我深入研究了如何在传统的关系数据库表结构中存储和操作树形数据。通过设计包含 `id`、`data`、`pid` 三个字段的线性表结构，我实现了树形层次关系在平面表中的映射存储。核心算法部分，我编写了 `linear_to_tree` 函数来完成 $O(n)$ 时间复杂度的线性表到树结构转换，该算法通过两遍扫描分别创建节点对象和建立父子关系，最终构建出完整的内存树结构。同时实现了 `insert_tree_node` 和 `delete_tree_node` 两个关键算法，前者支持在指定父节点下插入新节点并自动维护 ID 分配和索引更新，后者则实现了级联删除功能，能够递归删除目标节点及其所有子节点，确保数据一致性。

在技术架构层面，我采用 Flask 框架构建了完整的 Web 应用系统。后端通过 RESTful API 设计，提供了 `/api/table`、`/api/tree`、`/api/node` 等核心接口，支持获取线性表数据、获取树结构数据以及节点的增删操作。前端部分我使用原生 HTML、CSS 和 JavaScript 开发了双视图展示界面，左侧实时显示线性表的表格形式，右侧同步展示对应的树形结构，两个视图能够实时联动更新。特别是在交互设计上，我实现了父节点下拉选择器的动态更新、节点删除的确认机制以及数据刷新的实时同步，让整个系统具备了良好的用户体验。

3.2.总结与收获

这次设计让我把课本里的数据结构与数据库建模真正落到了“可运行、可验证、可视化”的系统里。关系数据库部分，我第一次切身感受到“逻辑结构与物理存储”的取舍：同一棵树既可以用关系表表达，又要在接口层保障插入、删除的一致性与复杂度；哈希索引的引入、双遍构建的 $O(n)$ 思路、根节点保护和级联删除的约束。B-Tree 部分我尽力把每个递归分支都写清楚、可回放、可追踪，完成了“操作历史 + 统计信息 + 结构导出”的组合校验。综上，我的自学能力在问题分解、资料检索、方案权衡、数据结构处理与 html 语言上都有明显提升——当看到树一层层长起来那种从抽象到具象的成就感，正是我最珍贵的收获。

第四部分 参考文献

- [1] 严蔚敏, 吴伟民. 数据结构(C 语言版)[M]. 北京: 清华大学出版社, 2007.
- [2] 王珊, 萨师煊. 数据库系统概论(第 5 版)[M]. 北京: 高等教育出版社, 2014.
- [3] 刘欢. HTML5 与 CSS3 基础教程(第 8 版)[M]. 北京: 人民邮电出版社, 2013.
- [4] 弗拉纳根. JavaScript 权威指南(第 6 版)[M]. 北京: 机械工业出版社, 2012.
- [5] 陈惠贞, 陈俊荣. HTML5+CSS3+JavaScript 网页设计入门必读[M]. 北京: 电子工业出版社, 2014.
- [6] 蜗牛学院, 邓强, 卿淳俊. Python Web 项目开发实战教程[M]. 人民邮电出版社: 202107: 249.
- [7] 陈翠琴. 基于 HTML5 技术的移动 Web 前端设计研究[J]. 信息记录材料, 2025, 26(09): 194-196. DOI: 10.16009/j.cnki.cn13-1295/tq.2025.09.002.
- [8] 廖黄鹏. 浅析 HTML5+CSS3 在网页设计中的特性及优势[J]. 信息与电脑, 2025, 37(01): 135-137.
- [9] 王晓雷, 王钱庆, 王鲜芳. 基于 Flask 数据可视化的网页端显示方法研究[J]. 无线互联科技, 2024, 21(15): 10-13+20.
- [10] Duckett J. HTML and CSS: Design and Build Websites[M]. Indianapolis: John Wiley & Sons, 2011.
- [11] [Flask 教程 | 菜鸟教程 https://www.runoob.com/flask/flask-tutorial.html](https://www.runoob.com/flask/flask-tutorial.html)
- [12] [Python 基础教程 | 菜鸟教程 https://www.runoob.com/python/python-tutorial.html](https://www.runoob.com/python/python-tutorial.html)
- [13] [svg 实现图形编辑器系列六: 连接线、连接桩思维导图、流程图等都有连接线的功能, 连接线可以在形状发生变化时自动跟随。本文会 - 掘金 https://juejin.cn/post/7213379884692684860](https://juejin.cn/post/7213379884692684860)